

ЛЕКЦИИ 8-9

SQL-99: дефиниция на схема,
ограничения, заявки и изгледи



5th Edition

Elmasri / Navathe

Дефиниция на данни, ограничения и промени в схемата

- Използват се за CREATE, DROP и ALTER на описанията на таблиците (релациите)на БД

CREATE TABLE

- Задава нова релация в БД чрез задаване на името и атрибутите ѝ и типовете данни (INTEGER, FLOAT, DECIMAL(i,j), CHAR(n), VARCHAR(n))
- Ограничение NOT NULL може да се задава за конкретен атрибут

```
CREATE TABLE DEPARTMENT (  
    DNAME          VARCHAR(10)      NOT NULL,  
    DNUMBER        INTEGER          NOT NULL,  
    MGRSSN         CHAR(9) ,  
    MGRSTARTDATE   CHAR(9)    ) ;
```

CREATE TABLE

- Командата се поддържа с вариации в SQL Server, MySQL, Oracle и PostgreSQL.
- Синтаксис според SQL-99:

```
CREATE [GLOBAL TEMPORARY | LOCAL  
TEMPORARY] TABLE table_name  
[ON COMMIT {PRESERVE ROWS | DELETE ROWS}  
(column_name datatype attributes [...n]  
| [LIKE table_name]  
| [table_constraint][,...n] ]
```

CREATE TABLE

- SQL99 *CREATE TABLE* създава *TEMPORARY* таблици, които се инстанцират при създаване и автоматично се унищожават в края на текущата сесия.
- Временните таблици могат да са *GLOBAL* и достъпни за всички активни потребители или *LOCAL* – достъпни са само за текущия потребител.
- Опцията *ON COMMIT PRESERVE ROWS* предпазва данните от модификация при *COMMIT*,
- *ON COMMIT DELETE ROWS* изчиства таблицата след *COMMIT*.
- Опцията *LIKE table_name* създава нова таблица със същата дефиниция на колони и ограничения, както на оригиналната. При избор на *LIKE*, не се налага дефиниране на ограничения върху колони или таблици.

CREATE TABLE

- В SQL2, можем да ползваме командата CREATE TABLE за задаване на атрибутите на първичния ключ, вторични ключове и ограничения върху референтната цялост (външни ключове).
- Ключовите атрибути могат да се зададат чрез фразите PRIMARY KEY и UNIQUE

```
CREATE TABLE DEPT (  
    DNAME          VARCHAR(10)      NOT NULL,  
    DNUMBER        INTEGER          NOT NULL,  
    MGRSSN         CHAR(9) ,  
    MGRSTARTDATE   CHAR(9) ,  
    PRIMARY KEY (DNUMBER) ,  
    UNIQUE (DNAME) ,  
    FOREIGN KEY (MGRSSN) REFERENCES EMP ) ;
```

DROP TABLE

- Използва се за изтриване на релация и нейната дефиниция.
- Релацията не може да се ползва повече в заявки, актуализации и други команди.
- Командата се поддържа с вариации в SQL Server, MySQL, Oracle и PostgreSQL. Синтаксис според SQL-99:

DROP TABLE table_name RESTRICT | CASCADE

- *RESTRICT* забранява изпълнението на командата, ако има референции към таблицата.
- *CASCADE* изпълнява каскадно изтриване на всички рефериращи обекти.
- Пример:

DROP TABLE DEPENDENT;

ALTER TABLE

- Използва се за актуализация на схемата на таблица. Командата се поддържа с вариации в SQL Server, Oracle и PostgreSQL, и с ограничения в MySQL. Синтаксис според SQL-99:

```
ALTER TABLE table_name
[ADD [COLUMN] column_name datatype attributes]
| [ALTER [COLUMN] column_name SET DEFAULT
default_value]
| [ALTER [COLUMN] column_name DROP DEFAULT]
| [ALTER [COLUMN] column_name ADD SCOPE table_name]
| [ALTER [COLUMN] column_name DROP SCOPE {RESTRICT |
CASCADE}]
| [DROP [COLUMN] column_name {RESTRICT | CASCADE}]
| [ADD table_constraint_name]
| [DROP CONSTRAINT table_constraint_name {RESTRICT |
CASCADE}]
```


ALTER TABLE

- Пример: Добавя атрибути към една релация
 - Новият атрибут ще има стойност NULL във всички tuples на релацията след изпълнение на командата. Поради това не се допуска използване на ограничение NOT NULL за този атрибут.

ALTER TABLE EMPLOYEE ADD JOB VARCHAR(12) ;

- След тази команда потребителят на БД може да добави стойности за новия атрибут JOB за всеки EMPLOYEE tuple.
 - Тази възможност се осъществява от командата UPDATE.

Добавени възможности в SQL2 и SQL-99

- Create schema
- Referential integrity options

CREATE SCHEMA

- Задава нова схема на БД, която се идентифицира с име. Поддържа се в SQL Server и Oracle (с вариации).
- Не се поддържа в MySQL и PostgreSQL.

```
CREATE SCHEMA [schema_name] [AUTHORIZATION  
owner_name]  
[DEFAULT CHARACTER SET char_set_name]  
[PATH schema_name [,...n] ]  
[ <create_table_statement1> [...n] ]  
[ <create_view_statement1> [...n] ]  
[ <grant statement1> [...n] ]
```

CREATE SCHEMA

- Пример от Oracle:

```
CREATE SCHEMA AUTHORIZATION emily
GRANT SELECT, INSERT ON view_1 TO sarah
GRANT ALL ON table_1 TO sarah
CREATE VIEW view_1 AS
    SELECT column_1, column_2
    FROM table_1
    ORDER BY column_2
CREATE TABLE table_1(column_1 INT,
    column_2 CHAR(20));
```

REFERENTIAL INTEGRITY OPTIONS (1)

- Можем да зададем RESTRICT, CASCADE, SET NULL или SET DEFAULT като ограничения върху референтната цялост (външни ключове)

```
CREATE TABLE DEPT (  
    DNAME          VARCHAR(10) NOT NULL,  
    DNUMBER        INTEGER          NOT NULL,  
    MGRSSN         CHAR(9) ,  
    MGRSTARTDATE   CHAR(9) ,  
    PRIMARY KEY (DNUMBER) ,  
    UNIQUE (DNAME) ,  
    FOREIGN KEY (MGRSSN) REFERENCES EMP  
    ON DELETE SET DEFAULT ON UPDATE CASCADE) ;
```

REFERENTIAL INTEGRITY OPTIONS (2)

```
CREATE TABLE EMP (  
    ENAME          VARCHAR(30)      NOT NULL,  
    ESSN           CHAR(9) ,  
    BDATE          DATE ,  
    DNO            INTEGER  DEFAULT 1 ,  
    SUPERSSN       CHAR(9) ,  
    PRIMARY KEY    (ESSN) ,  
    FOREIGN KEY    (DNO) REFERENCES DEPT  
        ON DELETE SET DEFAULT ON UPDATE  
        CASCADE ,  
    FOREIGN KEY    (SUPERSSN) REFERENCES EMP  
        ON DELETE SET NULL ON UPDATE CASCADE) ;
```

Допълнителни типове данни в SQL2 и SQL-99 (1)

DATE, TIME, TIMESTAMP

- **DATE:**

- Задава се във вида yyyy-mm-dd

- **TIME:**

- Задава се във формата hh:mm:ss

- **TIME(i):**

- Добавя и части от секундата като форматът е hh:mm:ss:ii...i

Допълнителни типове данни в SQL2 и SQL-99 (2)

- **TIMESTAMP:**

- Има DATE и TIME компоненти

- **INTERVAL:**

- Задава относителна стойност
- Може да бъде DAY/TIME или YEAR/MONTH интервал
- Може да е положително или отрицателно число. Когато се събира или изважда от абсолютна стойност, резултатът е абсолютна стойност.

Retrieval Queries in SQL (1)

- SQL има базов оператор за извличане на информация от БД – операторът **SELECT**
 - Той *не съвпада* със SELECT операция от релационната алгебра
- Важна разлика между SQL и релационния модел:
 - SQL позволява една таблица (релация) да има ≥ 2 еднакви tuples по всички атрибути
 - Следователно, SQL релация (таблица) е **multi-set** (или **bag**) от tuples; тя *не* е множество от tuples
- SQL релациите могат да се ограничат да бъдат множества, чрез задаване на атрибути PRIMARY KEY или UNIQUE, или чрез използване на опция DISTINCT в заявката

Retrieval Queries in SQL (2)

- **bag** или **multi-set** е като множество, но елементите могат да се повтарят >1 .
 - Пример: $\{A, B, C, A\}$ е bag. $\{A, B, C\}$ е bag и set.
 - Bags приличат на lists, но при bag няма ред на елементите.
- Пример:
 - $\{A, B, A\} = \{B, A, A\}$ са bags
 - Но, $[A, B, A]$ не е еквивалентно на $[B, A, A]$ като списъци

Retrieval Queries in SQL (3)

- Основната форма на SQL SELECT оператор е:

SELECT	<attribute list>
FROM	<table list>
WHERE	<condition>

- <attribute list> е списък от имена на атрибути, чиито стойности се извличат от заявката
- <table list> е списък от имена на релации, необходими за обработка на заявката
- <condition> е условен (Boolean) израз, който дефинира кои tuples да се върнат от заявката
- Командата се поддържа с вариации в SQL Server (ANSI join се поддържа напълно), MySQL (ANSI join се поддържа частично), Oracle (ANSI join не се поддържа) и PostgreSQL (ANSI join се поддържа напълно).

Синтаксис на SELECT в SQL-99

```
SELECT [ALL | DISTINCT] select_list  
FROM table_name1 [,..., table_nameN]  
[JOIN join_condition]  
[WHERE search_condition]  
[GROUP BY group_by_expression]  
[HAVING search_condition]  
[ORDER BY order_expression [ASC | DESC] ]
```

Синтаксис на SELECT

- SQL Server, MySQL и Oracle позволяват изчисление на математически изрази:

```
--QUERY (Microsoft, MySQL)  
SELECT 2 + 2
```

```
--QUERY (Oracle)  
SELECT 2 + 2  
FROM dual
```

Релационна схема на БД

EMPLOYEE

FNAME	MINIT	LNAME	<u>SSN</u>	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
-------	-------	-------	------------	-------	---------	-----	--------	----------	-----

DEPARTMENT

DNAME	<u>DNUMBER</u>	MGRSSN	MGRSTARTDATE
-------	----------------	--------	--------------

DEPT_LOCATIONS

<u>DNUMBER</u>	<u>DLOCATION</u>
----------------	------------------

PROJECT

PNAME	<u>PNUMBER</u>	PLOCATION	DNUM
-------	----------------	-----------	------

WORKS_ON

<u>ESSN</u>	<u>PNO</u>	HOURS
-------------	------------	-------

DEPENDENT

<u>ESSN</u>	<u>DEPENDENT_NAME</u>	SEX	BDATE	RELATIONSHIP
-------------	-----------------------	-----	-------	--------------

Попълнената БД

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alice	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Belkair, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmed	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS					DNUMBER	DLOCATION
					1	Houston
					4	Stafford
					5	Belkair
					5	Sugarland
					5	Houston

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1961-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Belkair	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1989-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1989-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

Прости SQL заявки (1)

- Основните SQL заявки съответстват на използване на следните операции от релационната алгебра:
 - SELECT
 - PROJECT
 - JOIN
- Всички примери използват БД COMPANY.

Прости SQL заявки (2)

- Пример: проста заявка върху една релация
- Query 0: Да се получи рожденната дата и адреса на служител на име 'John B. Smith'.

```
Q0: SELECT      BDATE, ADDRESS  
      FROM      EMPLOYEE  
      WHERE     FNAME='John' AND MINIT='B'  
      AND       LNAME='Smith'
```

- Подобно на SELECT-PROJECT операция от релационната алгебра:
 - Клаузата SELECT задава декартово произведение на атрибутите, като с WHERE клауза се показват условията на избор.
- Резултатът може да съдържа дублиращи се tuples.

Прости SQL заявки (3)

- Query 1: Да се получат имената и адресите на всички служители, които работят в отдела 'Research'.

```
Q1: SELECT      FNAME, LNAME, ADDRESS  
      FROM      EMPLOYEE, DEPARTMENT  
      WHERE     DNAME='Research' AND  
                DNUMBER=DNO
```

- Подобно на SELECT-PROJECT-JOIN последователност от релационната алгебра операциите:
- (DNAME='Research') е условие за избор (съответства на операция SELECT в релационната алгебра)
- (DNUMBER=DNO) е условие join (съответства на операция JOIN от релационната алгебра)

Прости SQL заявки (4)

- Query 2: За всеки проект, локализиран в 'Stafford', отпечатайте номера на проекта, номера на управляващия отдел, фамилията, адреса и датата на раждане на мениджъра на отдела.

```
Q2: SELECT  PNUMBER, DNUM, LNAME, BDATE, ADDRESS  
        FROM    PROJECT, DEPARTMENT, EMPLOYEE  
        WHERE    DNUM=DNUMBER AND MGRSSN=SSN  
                AND PLOCATION='Stafford'
```

- В Q2 има две join условия
- join условие DNUM=DNUMBER свързва проекта с управляващия го отдел
- join условие MGRSSN=SSN свързва управляващия отдел със служителя-мениджър на отдела

Aliases, * и DISTINCT, празна WHERE клауза

- В SQL можем да използваме едно и също име за ≥ 2 атрибута, когато атрибутите са в *различни релации*
- Една заявка, отнасяща се до ≥ 2 атрибута с еднакви имена, трябва да *квалифицира* името на атрибута с името на релацията, което ползва като *префикс* на името на атрибута.
- Пример:
- **EMPLOYEE.LNAME, DEPARTMENT.DNAME**

ALIASES (1)

- Някои заявки изискват рефериране на една релация два пъти
 - В този случай се използва *aliases* на името на релацията
- Query 8: За всеки служител, отпечатайте името на служителя и името на неговия пряк началник.

```
Q8:  SELECT      E.FNAME, E.LNAME, S.FNAME, S.LNAME
      FROM        EMPLOYEE E S
      WHERE       E.SUPERSSN=S.SSN
```

- В Q8 се ползват алтернативни имена на релацията EMPLOYEE E и S. Те се наричат *aliases* или *tuple variables*.
- Считаме E и S като две независими копия на EMPLOYEE; E представя служителите в ролята на *подчинени*, а S – като *началници*

ALIASES (2)

- Псевдонимите могат да се ползват в произволна SQL заявка за удобство.
- За същата цел може да се ползва ключовата дума AS:

```
Q8: SELECT      E.FNAME, E.LNAME,  
                S.FNAME, S.LNAME  
      FROM      EMPLOYEE AS E,  
                EMPLOYEE AS S  
      WHERE     E.SUPERSSN=S.SSN
```

Незададена *WHERE* клауза (1)

- Липсваща *WHERE* клауза означава „няма условие“; следователно всички tuples на релацията от клаузата *FROM* ще се върнат като резултат
 - Това е еквивалент на условието *WHERE TRUE*
- Query 9: Отпечатайте SSN за всички служители.
 - Q9:SELECT SSN
 FROM EMPLOYEE
- Ако е зададена >1 релации в клаузата *FROM* и *няма* зададено join условие, то се връща *ДЕКАРТОВОТО ПРОИЗВЕДЕНИЕ* на tuples.

Незададена WHERE клауза (2)

- Пример:

```
Q10: SELECT      SSN, DNAME  
      FROM        EMPLOYEE, DEPARTMENT
```

- От изключителна важност е да не се пропусне задаването на селекция и условие join в клаузата WHERE; в противен случай се връща некоректен и огромен резултат.

Използване на *

- За получаване на всички стойности на атрибутите на зададени tuples, се ползва *. Тя означава *всички атрибути*.

Примери:

Q1C:	SELECT	*
	FROM	EMPLOYEE
	WHERE	DNO=5

Q1D:	SELECT	*
	FROM	EMPLOYEE, DEPARTMENT
	WHERE	DNAME='Research' AND
		DNO=DNUMBER

Използване на DISTINCT

- SQL не третира релацията като множество; могат да съществуват дублиращи tuples.
- За елиминиране на дублираните tuples в резултата на заявката, се използва ключова дума **DISTINCT**.
- Пример: резултатът на Q11 може да съдържа дублирана заплата (SALARY), докато в Q11A това е премахнато.

Q11:	SELECT	SALARY
	FROM	EMPLOYEE
Q11A:	SELECT	DISTINCT SALARY
	FROM	EMPLOYEE

Операции върху множества (1)

- Някои от операциите върху множества са вградени в SQL. Дефинирани са: обединение (UNION) и, в *някои версии* на SQL, разлика (MINUS) и сечение (INTERSECT).
- Релацията резултат е множество от tuples; *дублираните tuples са премахнати от резултата.*
- Операциите върху множества се прилагат само за *обединение на съвместими резултати*; двете релации трябва да имат еднакви и в един и същи ред атрибути.

Операции върху множества (2)

- Query 4: Съставете списък на всички номера на проекти, с участник с фамилия 'Smith' от управляващия отдел на проекта.

```
Q4:      (SELECT      PNAME
          FROM        PROJECT, DEPARTMENT,
                      EMPLOYEE
          WHERE        DNUM=DNUMBER AND
                      MGRSSN=SSN AND LNAME='Smith')

          UNION

          (SELECT      PNAME
          FROM        PROJECT, WORKS_ON, EMPLOYEE
          WHERE        PNUMBER=PNO AND
                      ESSN=SSN AND NAME='Smith')
```

Вложени заявки (1)

- Пълна SELECT заявка, наречена *nested query*, може да бъде зададена в клаузата WHERE на друга заявка, която е *outer query*
 - Много от предходните примери могат да бъдат зададени с вложена заявка
- Query 1: Да се получат имената и адресите на всички служители от отдела 'Research'.

```
Q1:  SELECT      FNAME, LNAME, ADDRESS
      FROM        EMPLOYEE
      WHERE       DNO IN (SELECT DNUMBER
      FROM        DEPARTMENT
      WHERE       DNAME='Research' )
```

Вложени заявки (2)

- Вложената заявка избира номера на отдела 'Research'
- Външната заявка избира EMPLOYEE tuple ако неговата DNO стойност е в резултата на вложената заявка.
- Операцията IN сравнява стойност v с множество (или multi-set) от стойности V и връща TRUE, ако v е елемент от V
- Можем да използваме множество нива на влагане.
- Една референция към *неквалифициран атрибут* се отнася към релацията, декларирана на *най-вътрешно ниво*.

Свързани вложени заявки (1)

- Ако условието в клауза WHERE на *вложената заявка* реферира атрибут на релация, деклариран във *външна заявка*, двете заявки са *свързани*.
 - Резултатът от свързана вложена заявка е различен за всеки tuple (или комбинация от tuples) на релация(и) във външната заявка.
- Query 12: Да се получат имената на служителите със съвпадащо име с член на семейството си.

```
Q12: SELECT      E.FNAME, E.LNAME
      FROM
      WHERE
                  E.SSN IN
                        (SELECT      ESSN
                        FROM          DEPENDENT
                        WHERE          ESSN=E.SSN AND
                        E.FNAME=DEPENDENT NAME)
```

Свързани вложени заявки (2)

- В Q12 вложената заявка има различен резултат за всеки ред от външната заявка.
- Когато заявката е написана с вложен SELECT... FROM... WHERE... и използва сравнение = или IN, **винаги** се изпълнява като единична заявка. Например, Q12 може да бъде записана и като Q12A

Q12A:	SELECT	E.FNAME, E.LNAME
	FROM	EMPLOYEE E, DEPENDENT D
	WHERE	E.SSN=D.ESSN AND
		E.FNAME=D.DEPENDENT_NAME

Свързани вложени заявки (3)

- Оригиналният SQL, както е зададен в SYSTEM R, съдържа и оператор за сравнение **CONTAINS**, който се ползва заедно с вложените свързани заявки.
 - Този оператор е *отпаднал от езика*, заради трудности в ефективната му реализация.
 - Повечето реализации на SQL нямат такъв оператор.
 - Операторът CONTAINS сравнява *две множества от стойности* и връща TRUE, ако едното множество съдържа всички стойности от другото.
 - Напомня на операция подмножество от алгебрата.

Свързани вложени заявки (4)

- Query 3: Да се получат имената на служителите, които работят по всички проекти, управлявани от отдел с номер 5.

```
Q3:SELECT      FNAME, LNAME
      FROM      EMPLOYEE
      WHERE (    (SELECT      PNO
                   FROM        WORKS_ON
                   WHERE        SSN=ESSN)
              CONTAINS
              (SELECT      PNUMBER
                   FROM      PROJECT
                   WHERE      DNUM=5) )
```

Свързани вложени заявки (5)

- В Q3 втората вложена заявка, *която не е свързана с външната заявка*, извлича номерата на всички проекти, управлявани в отдел номер 5.
- Първата вложена заявка, която е свързана, извлича номерата на проектите, по които служителя работи, които са *различни за всеки служител*.

Функция EXISTS (1)

- EXISTS проверява дали резултата от вложена заявка не е празно множество
 - Можем да променим Query 12 в Q12B като използваме EXISTS.

Функция EXISTS (2)

- Query 12: Да се получат имената на служителите със съвпадащо име с член на семейството си.

```
Q12B:      SELECT      FNAME, LNAME
            FROM        EMPLOYEE
            WHERE        EXISTS
                        (SELECT      *
                         FROM        DEPENDENT
                         WHERE        SSN=ESSN
                         AND
                         FNAME=DEPENDENT_NAME)
```

Функция EXISTS (3)

- Query 6: Да се получат имената на служителите, които нямат семейство.

```

Q6:SELECT      FNAME, LNAME
      FROM      EMPLOYEE
      WHERE     NOT EXISTS
                (SELECT      *
                  FROM        DEPENDENT
                  WHERE       SSN=ESSN)
  
```

- В Q6 свързаната вложена заявка извлича всички DEPENDENT tuples, които са семейство с текущия EMPLOYEE tuple. Ако *резултатът е празно множество*, се избира текущия EMPLOYEE tuple.
 - EXISTS изразява мощта на SQL.

Работа с явни множества

- Възможно е да се ползват **явни (изброени) множества от стойности** в клаузата WHERE, вместо вложена заявка.
- Query 13: Да се получат номерата на всички служители, които работят по проекти 1, 2 или 3.

```
Q13: SELECT      DISTINCT      ESSN  
      FROM        WORKS_ON  
      WHERE       PNO IN (1, 2, 3)
```

NULL в SQL заявки

- SQL позволява проверка за **NULL** в заявките.
- SQL ползва **IS** или **IS NOT** за тази цел, защото се счита, че стойност NULL е различна от другите стойности, както и че две стойности NULL *не са еднакви*.
- Query 14: Да се получат имената на всички служители, които нямат преки началници.

```
Q14: SELECT      FNAME, LNAME  
      FROM        EMPLOYEE  
      WHERE       SUPERSSN IS NULL
```

- Ако е зададено условие join, tuples със стойности NULL на join атрибута не се включват в резултата.

Клауза JOIN

- В реализацията non-ANSI standard, операцията join се изпълнява в клауза *WHERE*. В ANSI SQL-92 стандарт, join се изпълнява в клауза *JOIN* на заявката.
- Тези методи join са познати като *theta style* и *ANSI style* of joins, съответно.

- Пример:

```
SELECT E.ESSN, E.FNAME, E.LNAME, J.JOB_DESC  
FROM EMPLOYEE AS E  
JOIN JOBS AS J ON E.JOB_ID = J.JOB_ID  
ORDER BY E.FNAME, E.LNAME
```

Типове JOIN (1)

- *Cross Join* – задава декартово произведение на 2 таблици. Поддържа се от SQL Server.

-- theta style

```
SELECT      E.ESSN, E.FNAME, E.LNAME, J.JOB_DESC  
FROM        EMPLOYEE E, JOBS J
```

-- ANSI style

```
SELECT      E.ESSN, E.FNAME, E.LNAME, J.JOB_DESC  
FROM        EMPLOYEE E  
CROSS JOIN  JOBS J
```

Типове JOIN (2)

- *Inner Join* – редовете, които нямат връзка в другата таблица, се унищожават. Този тип join е по подразбиране в ANSI style (поддържа се от Microsoft SQL Server, PostgreSQL и MySQL в момента):

-- theta style

```
SELECT E.ESSN,E.FNAME,E.LNAME,J.JOB_DESC  
FROM EMPLOYEE E, JOBS J  
WHERE E.JOB_ID = J.JOB_ID
```

-- ANSI style

```
SELECT E.ESSN,E.FNAME,E.LNAME,J.JOB_DESC  
FROM EMPLOYEE E  
JOIN JOBS J ON E.JOB_ID = J.JOB_ID
```

Типове JOIN (3)

- *Left [Outer] Join* – връщат се всички записи от лявата таблица в оператора join. Ако няма съответстващ запис отдясно, полетата са NULL. Повечето професионалисти препоръчват конфигуриране на outer joins като left joins при възможност (поддържа се от Microsoft SQL Server в момента).

Типове JOIN (4)

-- Oracle theta style

```
SELECT E.ESSN,E.FNAME,E.LNAME,J.JOB_DESC  
FROM EMPLOYEE E, JOBS J  
WHERE J.JOB_ID (+) = E.JOB_ID
```

-- ANSI style

```
SELECT E.ESSN,E.FNAME,E.LNAME,J.JOB_DESC  
FROM EMPLOYEE E  
LEFT JOIN JOBS J ON E.JOB_ID = J.JOB_ID
```

Типове JOIN (5)

- *Right [Outer] Join* - връщат се всички записи от дясната таблица в оператора join. Ако няма съответстващ запис отляво, полетата са NULL (поддържа се от Microsoft SQL Server в момента).

-- Oracle theta style

```
SELECT E.ESSN,E.FNAME,E.LNAME,J.JOB_DESC  
FROM EMPLOYEE E, JOBS J  
WHERE J.JOB_ID = (+) E.JOB_ID
```

-- ANSI style

```
SELECT E.ESSN,E.FNAME,E.LNAME,J.JOB_DESC  
FROM EMPLOYEE E  
RIGHT JOIN JOBS J ON E.JOB_ID = J.JOB_ID
```

Типове JOIN (6)

- *Full Join* – задава връщане на всички записи от двете таблици, независимо дали имат съответен запис в другата таблица. При липса на данни полетата се попълват с NULL (поддържа се от Microsoft SQL Server в момента):

-- theta style не поддържа тази функция

-- ANSI style

```
SELECT E.ESSN,E.FNAME,E.LNAME,J.JOB_DESC  
FROM EMPLOYEE E  
FULL JOIN JOBS J ON E.JOB_ID = J.JOB_ID
```

Multi-table Join пример

--theta style

```
SELECT a.au_lname, a.au_fname, t2.title  
FROM authors a, titleauthor t1, titles t2  
WHERE a.au_id = t1.au_id AND t1.title_id = t2.title_id  
ORDER BY t2.title
```

-- ANSI style

```
SELECT a.au_lname, a.au_fname, t2.title  
FROM authors a  
JOIN titleauthor AS t1 ON a.au_id = t1.au_id  
JOIN titles AS t2 ON t1.title_id = t2.title_id  
ORDER BY t2.title
```


Multi-key Join пример

--theta style

```
SELECT s1.store_id, s1.title_id, s2.qty  
FROM sales s1, sales_projections s2  
WHERE s1.store_id = s2.store_id AND s1.title_id = s2.title_id  
ORDER BY s1.store_id, s2.title_id
```

-- ANSI style

```
SELECT s1.store_id, s1.title_id, s2.qty  
FROM sales s1  
JOIN sales_projections s2 ON s1.store_id = s2.store_id  
AND s1.title_id = s2.title_id  
ORDER BY s1.store_id, s2.title_id
```

Характеристики на Join в SQL2 (1)

- Може да се зададе "joined relation" в клаузата FROM.
 - Изглежда като всяка друга релация, но е резултат на join
 - Позволява на потребителя да зададе различни типове joins (обикновен "theta" JOIN, NATURAL JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, CROSS JOIN и т.н.)

Характеристики на Join в SQL2 (2)

- Примери:

```
Q8:  SELECT      E.FNAME, E.LNAME, S.FNAME,  
              S.LNAME  
      FROM        EMPLOYEE E S  
      WHERE       E.SUPERSSN=S.SSN
```

- Може да бъде записан като:

```
Q8:  SELECT      E.FNAME, E.LNAME, S.FNAME,  
              S.LNAME  
      FROM        (EMPLOYEE E LEFT OUTER JOIN  
                   EMPLOYEES ON  E.SUPERSSN=S.SSN)
```

Характеристики на Join в SQL2 (3)

- Примери:

Q1: SELECT FNAME, LNAME, ADDRESS
FROM EMPLOYEE, DEPARTMENT
WHERE DNAME='Research' AND
DNUMBER=DNO

- Може да бъде записан като:

Q1: SELECT FNAME, LNAME, ADDRESS
FROM (EMPLOYEE JOIN DEPARTMENT
ON DNUMBER=DNO)
WHERE DNAME='Research'

- Или като:

Q1: SELECT FNAME, LNAME, ADDRESS
FROM (EMPLOYEE NATURAL JOIN
DEPARTMENT
AS DEPT(DNAME, DNO, MSSN,
MSDATE)
WHERE DNAME='Research'

Характеристики на Join в SQL2 (4)

- Query 2: За всеки проект, локализиран в 'Stafford', отпечатайте номера на проекта, номера на управляващия отдел, фамилията, адреса и датата на раждане на мениджъра на отдела.

```
Q2:  SELECT      PNUMBER, DNUM, LNAME, BDATE,
                ADDRESS
      FROM      PROJECT, DEPARTMENT, EMPLOYEE
      WHERE     DNUM=DNUMBER AND MGRSSN=SSN
                AND PLOCATION='Stafford'
```

- Q2 може да бъде записан както е показано по-долу; така се илюстрира множество joins в joined tables

```
Q2:  SELECT      PNUMBER, DNUM, LNAME,
                BDATE, ADDRESS
      FROM      (PROJECT JOIN
                DEPARTMENT ON
                    DNUM=DNUMBER) JOIN
                EMPLOYEE ON
                    MGRSSN=SSN) )
      WHERE     PLOCATION='Stafford'
```

AGGREGATE FUNCTIONS (1)

- Тук се включва **COUNT, SUM, MAX, MIN, AVG**
- Query 15: Да се получи максималната, минималната и средната заплата на всички служители.

```
Q15: SELECT      MAX(SALARY),  
                  MIN(SALARY), AVG(SALARY)  
FROM  EMPLOYEE
```

- Някои SQL реализации *може и да не позволяват повече от една функции* в клаузета SELECT.

AGGREGATE FUNCTIONS (2)

- Query 16: Да се получи максималната, минималната и средната заплата на всички служители, работещи в отдела 'Research'.

```
Q16: SELECT      MAX(SALARY),  
                 MIN(SALARY), AVG(SALARY)  
FROM  EMPLOYEE, DEPARTMENT  
WHERE      DNO=DNUMBER AND  
           DNAME='Research'
```

AGGREGATE FUNCTIONS (3)

- Queries 17 и 18: Да се изчисли общият брой служители в компанията (Q17), и броят на служителите в отдел 'Research' (Q18).

```
Q17: SELECT      COUNT (*)  
      FROM      EMPLOYEE
```

```
Q18: SELECT      COUNT (*)  
      FROM      EMPLOYEE, DEPARTMENT  
      WHERE      DNO=DNUMBER AND  
                  DNAME='Research'
```


Групиране (1)

- В много случаи желаем да приложим агрегатна функция към *подгрупи от of tuples* в релацията.
- Всяка подгрупа of tuples се състои от множество tuples, които *имат една и съща стойност на групиращия атрибут(u)*.
- Функцията се прилага за всяка подгрупа поотделно.
- За задаване на групиращи атрибути, SQL има клауза **GROUP BY**. Те трябва също да присъстват в клаузата **SELECT**.

Групиране (2)

- Query 20: За всеки отдел, да се получи номера на отдела, броя служители в него и средната им заплата.

```
Q20: SELECT      DNO, COUNT (*), AVG (SALARY)
      FROM        EMPLOYEE
      GROUP BY    DNO
```

- В Q20, EMPLOYEE tuples са групирани.
 - Всяка група има една и съща стойност на групиращия атрибут DNO.
- Функциите COUNT и AVG се прилагат за всяка група поотделно.
- Клаузата SELECT включва само групиращия атрибут и приложените функции към всяка група tuples.
- Условие join може да се използва заедно с групирането.

Групиране (3)

- Query 21: За всеки проект, да се получи номера, името и броя служители, работещи по него.

```
Q21: SELECT      PNUMBER, PNAME, COUNT (*)  
      FROM        PROJECT, WORKS_ON  
      WHERE       PNUMBER=PNO  
      GROUP BY    PNUMBER, PNAME
```

- В този случай групирането и функциите се прилагат след join на две релации.

Клауза HAVING (1)

- Понякога се налага извличане на данни и групирането им, само *ако групата отговаря на определени условия*.
- Клаузата **HAVING** се ползва за задаване на условие на селекцията в групите (а не на отделни tuples, както е в WHERE).

Клауза HAVING (2)

- Query 22: За всеки проект, *по който работят* >2 *служители*, да се получат номера, името и броя на служителите, работещи по него.

```
Q22:      SELECT      PNUMBER, PNAME, COUNT(*)  
           FROM        PROJECT, WORKS_ON  
           WHERE       PNUMBER=PNO  
           GROUP BY    PNUMBER, PNAME  
           HAVING      COUNT (*) > 2
```

Сравнение на поднизове (1)

- Операторът за сравнение **LIKE** се използва при сравнение на части от низове.
- Два резервирани символа се използват за целта:
 - '%' (понякога е заменен от '*') – замества произволен брой произволни по стойност символи.
 - '_' – замества един произволен символ.

Сравнение на поднизове (2)

- Query 25: Да се получат всички служители, чиито адрес е в Houston, Texas. Адресът ще се зададе с низ от вида: ,%Houston,TX%'.

```
Q25: SELECT      FNAME, LNAME  
      FROM        EMPLOYEE  
      WHERE       ADDRESS LIKE  
                  '%Houston,TX%'
```

Сравнение на поднизове (3)

- Query 26: Да се получат всички служители, родени през 1950 г.
 - Тук символа '5' трябва да бъде 8-мия подред (съгласно формата на датата) и стойността на BDATE трябва да бъде от вида '_____5_', като всяка долна черта замества един символ.

```
Q26: SELECT      FNAME, LNAME  
      FROM        EMPLOYEE  
      WHERE       BDATE LIKE '_____5_'
```

- Операторът LIKE позволява да се заобиколи факта, че всяка стойност е неделима и дава възможност да се обединят по някакъв признак.
 - Следователно, в SQL, символният низ не е атомарна стойност.

Аритметични операции

- Стандартните аритметични операции '+', '-', '*', '/' могат да се прилагат върху числови данни в резултата от SQL заявка.
- Query 27: Да се отпечата увеличението на заплатата с 10% на работещите по проекта 'ProductX'.

```
Q27: SELECT      FNAME, LNAME, 1.1*SALARY  
      FROM        EMPLOYEE, WORKS_ON,  
      PROJECT  
      WHERE       SSN=ESSN AND PNO=PNUMBER  
                  AND PNAME='ProductX'
```

ORDER BY (1)

- Клаузата **ORDER BY** се използва за сортиране на tuples в заявка, базирана на стойностите на няколко атрибута.
- Query 28: Да се отпечати списък на служителите и проектите, по които те работят, подреден по отдела на служителя. Във всеки отдел подредбата да е по азбучен ред на фамилията на служителите в него.

```
Q28: SELECT      DNAME, LNAME, FNAME, PNAME
      FROM        DEPARTMENT, EMPLOYEE,
                  WORKS_ON, PROJECT
      WHERE       DNUMBER=DNO AND SSN=ESSN
                  AND PNO=PNUMBER
      ORDER BY    DNAME, LNAME
```

ORDER BY (2)

- Редът на сортиране, по подразбиране, е нарастващ.
- Можем да зададем ключова дума **DESC**, ако желаем да променим реда в намаляващ. Ключовата дума **ASC** може да се ползва за явно задаване на нарастващ ред на сортиране.

Преглед на SQL заявките (1)

- Една заявка в SQL може да се състи от мах 6 клаузи, но само първите 2, SELECT и FROM, са задължителни. Клаузите се задават в следния ред:

SELECT	<attribute list>
FROM	<table list>
[WHERE	<condition>]
[GROUP BY	<grouping attribute(s)>]
[HAVING	<group condition>]
[ORDER BY	<attribute list>]

Преглед на SQL заявките (2)

- Клаузата SELECT представя атрибутите и функциите, които трябва да се покажат и изпълнят съответно.
- Клаузата FROM задава всички релации (или псевдоними), необходими в заявката, но не и за вложените заявки.
- Клаузата WHERE задава условията за избор и join на tuples от релациите, зададени в клаузата FROM.
- GROUP BY задава групиращите атрибути.
- HAVING указва условието за селекция на групите.
- ORDER BY задава реда на отпечатване на резултата от заявката.
 - Една заявка се изчислява първо от прилагане на клаузата WHERE, след това на GROUP BY и HAVING и накрая на клаузата SELECT.

Актуализации в SQL

- Съществуват три команди в SQL за модифициране на БД: **INSERT**, **DELETE**, **UPDATE**

INSERT (1)

- В най-простата форма INSERT добавя ≥ 1 tuples към една релация.
- Стойностите на атрибутите трябва да бъдат записани в същия ред, в който атрибутите са зададени в командата **CREATE TABLE**.
- Командата се поддържа с вариации от SQL Server, MySQL и Oracle и напълно от PostgreSQL.

```
INSERT [INTO] [[database_name.]owner.] {table_name |  
view_name} [(column_list)]  
{[DEFAULT] VALUES | VALUES (value[,...]) |  
SELECT_statement }
```

INSERT (2)

- Пример:

```
U1: INSERT INTO EMPLOYEE  
VALUES ('Richard','K','Marini', '653298653', '30-DEC-52',  
'98 Oak Forest,Katy,TX', 'M', 37000,'987654321', 4 )
```

- Една алтернативна форма на INSERT задава явно имената на атрибутите, които кореспондират със стойностите в новия tuple.

- Атрибутите със стойност NULL могат да се пропуснат.

- Пример: Добавете един tuple за нов EMPLOYEE, на който са известни само атрибутите FNAME, LNAME, и SSN.

```
U1A: INSERT INTO EMPLOYEE (FNAME, LNAME,  
SSN)  
VALUES ('Richard', 'Marini', '653298653')
```


INSERT (3)

- Само ограниченията, зададени в DDL команди се активират автоматично, когато се прилагат актуализации на БД.
 - Друга вариация на INSERT позволява вмъкване на *множество tuples*, като резултат от заявка в релация.

INSERT (4)

- Пример: Нека трябва да се създаде временна таблица, която да съдържа имената на отделите, броя на служителите във всеки от тях и сумарната заплата за всеки отдел.
 - Таблицата DEPTS_INFO се създава от U3A и се зарежда с информация от БД чрез заявка в U3B.

```
U3A: CREATE TABLE DEPTS_INFO
      (DEPT_NAME          VARCHAR(10),
       NO_OF_EMPS         INTEGER,
       TOTAL_SAL          INTEGER);
```

```
U3B: INSERT INTO DEPTS_INFO (DEPT_NAME,
                             NO_OF_EMPS, TOTAL_SAL)
      SELECT DNAME, COUNT (*), SUM (SALARY)
      FROM DEPARTMENT, EMPLOYEE
      WHERE DNUMBER=DNO
      GROUP BY DNAME ;
```

INSERT (5)

- Таблицата DEPTS_INFO може да не е актуална винаги, ако променим някой tuple в релации DEPARTMENT или EMPLOYEE след получаване на U3B. В такива случаи създаваме view за да поддържаме таблицата винаги актуална.

DELETE (1)

- Изтрива tuples от релация.
 - Включва клауза WHERE за избор на tuples, които трябва да се изтрият.
 - Задължително се проверява референтната цялост.
 - Tuples се изтриват *само от една таблица в даден момент* (освен ако не е зададена опция CASCADE в ограниченията за референтна цялост).
 - Липсваща клауза WHERE задава изтриване на *всички tuples* в релацията; таблицата става празна.
 - Броят на изтрите tuples зависи от броя на tuples в релацията, които удовлетворяват клаузата WHERE.
- Командата се поддържа с вариации от SQL Server и MySQL, и напълно от Oracle и PostgreSQL.

DELETE [FROM] [owner.]table_name [WHERE clause]

DELETE (2)

- Примери:

U4A: DELETE FROM
WHERE

EMPLOYEE
LNAME='Brown'

U4B: DELETE FROM
WHERE

EMPLOYEE
SSN='123456789'

U4C: DELETE FROM
WHERE

EMPLOYEE
DNO IN
(SELECT DNUMBER
FROM DEPARTMENT
WHERE
DNAME='Research')

U4D: DELETE FROM

EMPLOYEE

UPDATE (1)

- Използва се за модификация на стойност на атрибут в ≥ 1 избрани tuples.
- Клаузата WHERE избира кои tuples да бъдат актуализирани.
- Допълнителна клауза SET задава атрибутите, които ще се променят и новите им стойности.
- Всяка команда променя tuples *само в една релация*.
- Проверява се референтната цялост на БД.
- Командата се поддържа с вариации от SQL Server, MySQL и Oracle, и напълно от PostgreSQL.

UPDATE (2)

UPDATE {table_name | view_name}
SET {column_name | variable_name} = {DEFAULT |
expression} [,...n]
WHERE conditions

- Пример: Променете местоположението и управляващия отдел на проект No 10 съответно на 'Bellaire' и 5.

```
U5:  UPDATE      PROJECT  
      SET        PLOCATION = 'Bellaire',  
              DNUM = 5  
      WHERE      PNUMBER=10
```

UPDATE (3)

- Пример: Увеличете заплатата на служителите от отдел 'Research' с 10%.

```
U6: UPDATE      EMPLOYEE
      SET        SALARY = SALARY *1.1
      WHERE      DNO IN (SELECT  DNUMBER
                           FROM    DEPARTMENT
                           WHERE    DNAME='Research')
```

- В тази заявка, модифицираната стойност на заплатата зависи от оригиналната заплата във всеки ред.
 - Референцията към атрибута SALARY отлясно на = означава референция към старата стойност на SALARY преди промяната.
 - Референцията към атрибута SALARY отляво на = означава референция към новата стойност на SALARY след промяната.

Преглед на SQL заявки

- Една заявка в SQL може да съдържа до 6 клаузи, ако само SELECT и FROM са задължителни. Клаузите се задават в реда:

SELECT	<attribute list>
FROM	<table list>
[WHERE	<condition>
[GROUP BY	<grouping attribute(s)>
[HAVING	<group condition>
[ORDER BY	<attribute list>

- Съществуват три SQL команди за промяна на информацията в БД: **INSERT, DELETE, UPDATE**